

secp256k1 与密码技术的分析报告

实验题目	任务2：分析secp256k1库与密码技术的使用
实验日期	2026年7月20日
实验学时	4h
小组成员	张涵 / 彭炜霖 / 李庭跃

目录

- 安全哲学与常数时间设计
 - GLV endomorphism 争议史：不是"移除"而是"隔离"
 - 常数时间防御体系：从语义到掩码
- 性能演进路径
 - ecmult 与 ecmult_gen 的分离：公开与秘密的两条流水线
 - 有限域底层：5×52 表示与 BMI2/ADX 的兴衰
 - BIP340 Schnorr 与批量验证：MSM 的两次跃迁
- 数学原理与实现推导
 - Jacobian 投影坐标：用乘法消灭逆元
 - ECDSA 与 BIP340 Schnorr：构造、线性与归约
 - Tagged Hash 的数学动机
- 关键 Commit / PR 索引
- 结语

1：安全哲学与常数时间设计

1.1 GLV endomorphism 的争议史：不是"移除"，而是"隔离"

secp256k1 是一条 Koblitz 曲线 ($y^2 = x^3 + 7, a = 0$)，因此存在高效可计算自同态。设 β 为 $\mathbb{F}_p$ 中满足 $\beta^3 \equiv 1 \pmod{p}$ 的非平凡三次单位根， λ 为群阶 n 下满足 $\lambda^3 \equiv 1 \pmod{n}$ 的标量，则映射

$$\phi(x, y) = (\beta x, y) = [\lambda](x, y)$$

是一个仅消耗一次域乘法的点乘。

GLV 方法将 256 位标量 k 经格基约化（在 Eisenstein 整数环 $\mathbb{Z}[\zeta_3]$ 中做最近平面划分）分解为

$$k = k_1 + k_2\lambda$$

其中 $|k_1|, |k_2| \approx 2^{128}$ ，于是

$$kP = k_1P + k_2\phi(P)$$

配合 Shamir 联合倍乘，可将点乘的倍点次数近乎减半。

该优化的争议集中在三点：

争议维度	内容	工程决策
专利	GLV 专利 (Certicom, US7110538) 直至 2020-09 才到期	2014 年起以 USE_ENDOMORPHISM 编译宏默认关闭 (commit 399c03f "Make endomorphism optimization optional")
法律 vs 技术	到期后是否默认开启	PR #830 (2020-10) 直接删除非 endomorphism 代码路径, 使验证端无条件使用
侧信道	λ 分裂后的双标量路径是否引入新的时序面	只在 variable-time 的 ecmult (验证) 中使用; 常数时间的 ecmult_gen/ecmult_const (签名、ECDH) 从不使用

上表最后一行是本节的安全哲学核心: **同一库内, 优化准入门槛按秘密性分级。**

- **验证路径**处理的全是公开数据 (签名、公钥), 可以使用 wNAF (窗口值依赖数据)、GLV 分裂、提前退出等一切 variable-time 技巧。
- **签名路径**处理私钥与 nonce, 任何数据依赖的分支、内存访问、提前终止都被视为漏洞。

GLV 不是被"移除"了, 而是被**安全域隔离**了——分裂函数 `secp256k1_scalar_split_lambda` 虽然本身可写成无分支形式, 但它把标量映射到两个半宽标量后的 wNAF 编码长度和符号分布依赖秘密值; 且 comb 方法的固定表访问模式已足够快, 引入 GLV 的边际收益抵不上分析成本的指数级上升。

1.2 常数时间防御体系: 从语义到掩码

库的防御哲学可归纳为四层:

1. **无秘密依赖分支**: 标量运算 (`scalar_4x64_impl.h`) 全部用进位链 + 条件归约实现, 例如模 n 加法的"减 n 再按借位加回"模式, 用算术代替 if。
2. **无秘密依赖内存访问**: 所有查表 (`ecmult_gen` 的 comb 表、wNAF 表) 通过 `secp256k1_fe_cmov / secp256k1_fe_storage_cmov` 线性扫描全表并条件移动, 缓存访问轨迹与被选索引无关。
3. **无秘密依赖指令延迟**: 避免在秘密数据上使用除法、变长移位; 模逆用 `safegcd divsteps` 的固定迭代轮数版本 (PR #831、PR #906)。
4. **可验证性**: 测试框架在 Valgrind 下以"未初始化内存"标记秘密值 (ctgrind 式方法), 任何秘密流向分支谓词或地址计算都会在 CI 中报错; 另有 `tests_exhaustive.c` 在曲线扭线上的小阶子群 (阶 13、199) 上穷举所有群运算与签名, 捕获只在边界条件出现的代数错误。

`field_5x52_impl.h` 中 `secp256k1_fe_impl_cmov` 的核心逻辑 (伪代码):

```
/* flag ∈ {0,1}; flag=1 时 r := a, 否则 r 不变。无分支、无秘密索引 */
SECP256K1_INLINE static void fe_cmov(fe *r, const fe* a, int flag) {
    uint64_t mask = (uint64_t)0 - (uint64_t)flag; /* 0 -> 0x0, 1 -> ~0x0 */
    uint64_t nmask = ~mask;
    for (i = 0; i < 5; i++)
        r->n[i] = (r->n[i] & nmask) | (a->n[i] & mask);
}
```

查表时的使用模式是: 对表中每一个条目计算"索引是否等于目标"的掩码 (`cmov(tab[i], entry, i == idx)`), 整张表在时间上被均匀触摸, `cache_line` 访问序列与秘密索引正交。

群运算层则用统一/完备公式消除点级分支：secp256k1_gej_add_ge 内部对 $P = Q$ 、 $P = -Q$ 、无穷远点等退化情形不做提前分支，而是通过 infinity 标志位与掩码化的坐标选择走同一条算术路径（commit 5de4c5d 修复的正是 $P + (-\lambda)P$ 退化情形下的统一性问题）。

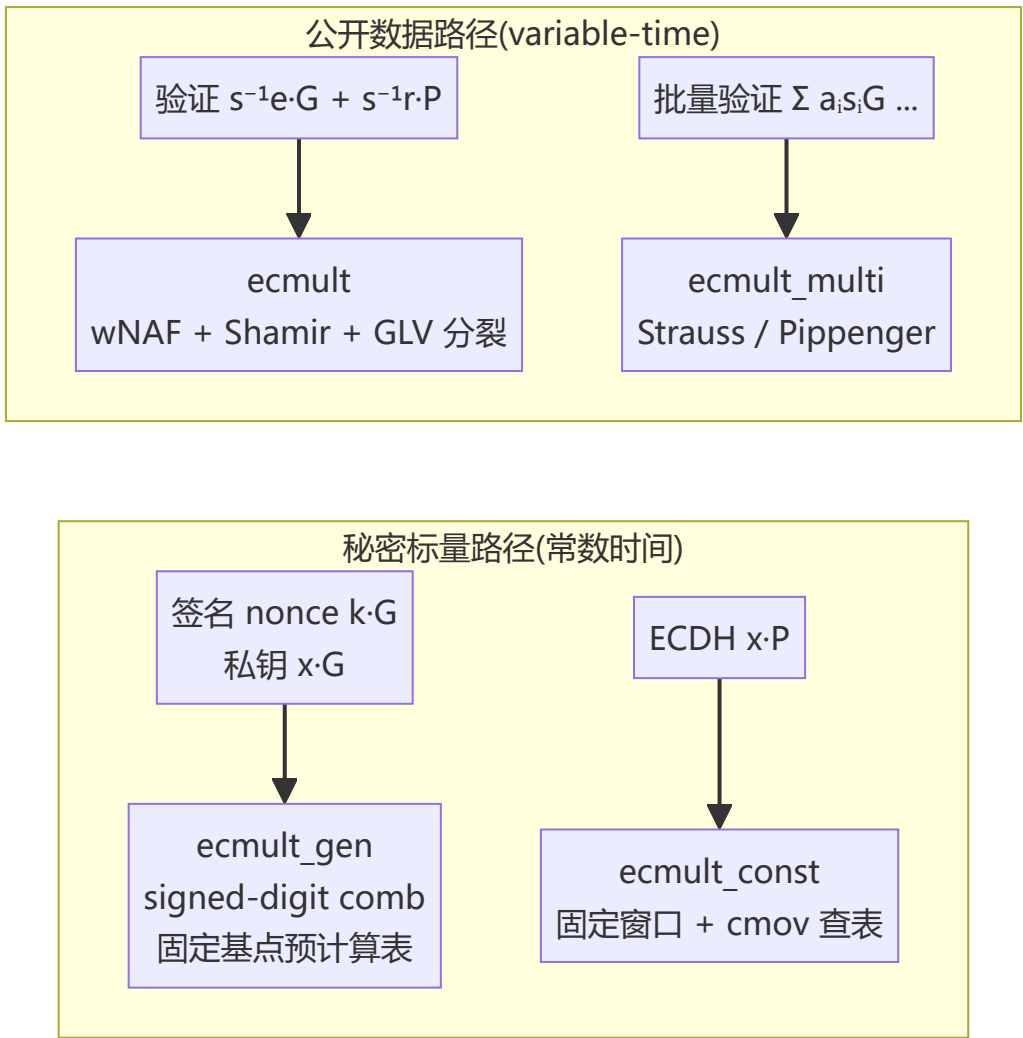
对差分功耗分析（DPA）这类能穿透“平均常数时间”的攻击，库提供可选的运行时盲化（见 2.1 节），其设计原则写在 README 中：

"预计算表中加入（最终再减去）若干离散对未知的点，即使攻击者控制私钥也无法控制内部数据。"

这是一种纵深防御：常数时间阻断时序/缓存通道，盲化挫败能量通道，二者正交。

2：性能演进路径

2.1 ecmult 与 ecmult_gen 的分离：公开与秘密的两条流水线



ecmult_gen (签名/密钥生成, $kGk^{**}G$)

利用基点固定这一事实，离线生成 $16^i G$ 系列的预计算表，在线计算退化为一系列点加。

2024 年合并的 PR #1058 将其重写为 signed-digit multi-comb。设 COMB_TEETH、COMB_SPACING、COMB_BLOCKS 满足：

$$TEETH \times SPACING \times BLOCKS \geq 256$$

标量 d 被写成带符号齿的表示：

$$d = \sum_{b=0}^{B-1} \sum_{t=0}^{T-1} s_{b,t}, 2^{(bT+t)S}, \quad s_{b,t} \in \pm 1, \pm 2, \dots$$

每个块查一张 2^{TEETH} 项的表 (cmov 访问)，外层循环按 `comb_off` 从 `SPACING-1` 降到 0 做倍点累加。带符号表示使正负项共享表项 (点取反在 Jacobian 下仅需取负 y, x 不变)，表体积减半。

表大小通过 `SECP256K1_ECMULT_GEN_KB` 可配置为 2/22/86 KiB，2024 年后桌面默认 86 KiB (issue #1562、PR #1564)。

盲化 (PR #57/#190 确立，现行实现见 `ecmult_gen_impl.h` 的 `secp256k1_ecmult_gen_blind`)：

上下文持有一个随机标量盲因子 b 与投影盲化值。计算被改写为：

$$R = (k - b), G + b, G$$

其中 $(k - b)G$ 走 `comb` 主路径， bG 由一张"起始点加盲"的机制吸收；此外每个查表结果在累加前用 `secp256k1_gej_rescale` 乘上随机 Z 坐标 (投影盲化)，使中间域元素的比特表示对功耗分析者不可预测。`secp256k1_context_randomize` 允许调用者周期性重播种盲因子。

`ecmult` (验证, $aP + bG$)

走完全相反的路线：variable-time wNAF (`WINDOW_A = 5` 用于 P 及其 GLV 同伴，`ECMULT_WINDOW_SIZE = 15` 用于 G 的大预计算表，默认约 $2^{13} \times 64$ 张表)，配合 Shamir 技巧逐位扫描四条 `wnaf` 序列 (a 分裂为 a_1, a_2 ， b 分裂为 b_1, b_2 ，每条约 129 位有效长度)，一次主循环完成四个半宽标量的联合倍乘。

2.2 有限域底层：5×52 表示与 BMI2/ADX 的兴衰

域大小：

$$p = 2^{256} - 2^{32} - 977$$

64 位平台采用 **5×52 饱和 limb 表示** (`field_5x52_int128_impl.h`)：

$$x = \sum_{i=0}^4 n_i, 2^{52i}$$

每个 limb 留出 12 位冗余延迟进位，乘法的 $64 \times 64 \rightarrow 128$ 中间积用 `unsigned __int128` 累加。模约化利用同余

$$2^{256} \equiv 2^{32} + 977 \pmod{p}$$

把最高 limb 溢出的部分 t 折叠回最低位：

$$n_0 += t \cdot 977, \quad n_1 += t \ll 32$$

至多两轮即可把幅值压回安全界。归一化 (`fe_normalize`) 与弱归一化分离，群运算大量消费"弱归一化"输入以省略完整进位传播——这是延迟归约 (lazy reduction) 的标准工程实践。

汇编层面，库曾长期携带 Peter Dettman 手写的 `yasm` 模块 `field_5x52_asm.asm` (x86_64)，其核心正是 BMI2/ADX 三件套：

- `mulx`：无标志位破坏的 $64 \times 64 \rightarrow 128$ 乘法，允许乘法流水线与进位链并行；
- `adcx / adox`：分别只使用 CF 与 OF 两条独立进位链，将 5×5 部分积的两条累加链交织调度，逼近 Intel 微架构的吞吐极限。

这段汇编的命运本身就是一部工程史：

- PR #1329 引入用 CryptOpt 超优化器生成的代码做对照实验；
- 最终 PR #1446 ([field: Remove x86_64 asm], 2023-12-01 合并) 将整个 x86_64 汇编删除。

现代 GCC/clang 对 `__int128` 的代码生成已与手写汇编持平甚至更优，而删除汇编消除了工具链（yasm 依赖）、可审计性与侧信道验证（Valgrind 无法标记汇编内秘密流）三方面的负担。这是“可验证性优先于微基准”哲学的又一次体现。

32 位平台则保留 `field_10x26`（10×26 limb），另有 Wladimir van der Laan 手写的 ARM 汇编 `field_10x26_arm.s`，标记为 experimental。

模逆的演进同样关键：

阶段	实现方式	问题
早期	GMP 的 <code>mpz_invert</code>	秘密数据进入变时间大数库——重大隐患
过渡	固定指数加法链（费马小定理 $x^{-1}=x^{p-2}$ ）	约 450 次域乘，较慢
当前	<code>safegcd</code> （Bernstein-Yang <code>divsteps</code> ），PR #831（2021-03）	复杂度降至约百次以内的 62 位“矩阵步”，且 PR #906 将 $\delta=1/2$ 的修正 <code>divsteps</code> 做到完全常数时间

GMP 依赖被彻底移除。标量（mod n）与域（mod p）共用 `modinv32` / `modinv64` 两套实现，并提供 `_var` 变时间变体供验证路径使用。

2.3 BIP340 Schnorr 与批量验证：MSM 的两次跃迁

PR #558（2020-09-11 合并）引入实现 BIP-340 的 `schnorr_sig` 模块（API 后经 PR #844 大改）。签名端成本与 ECDSA 相当（一次 `ecmult_gen` 加若干 tagged hash），但验证端解锁了**线性批量验证**，这才是性能跃迁的来源。

单个 Schnorr 验证为：

$$sG = R + eP$$

批量验证 n 个签名 $(P_i, m_i, (R_i, s_i))$ 时，取随机系数 $a_1 = 1, a_i = \text{hash}(\text{seed}|i) \ (i \geq 2)$ ，验证单一等式：

$$\left(\sum_{i=1}^n a_i s_i\right) G \stackrel{?}{=} \sum_{i=1}^n a_i R_i + \sum_{i=1}^n a_i e_i P_i$$

右边是一个 $2n$ 项的**多标量乘法（MSM）**。

仓库为此演进的分层如下：

阶段	PR	内容
Strauss 推广	commit 8c1c831（2016）	<code>ecmult_strauss_wnaf</code> 支持多点，逐位窗口联合扫描， $O(n)$ 内存

阶段	PR	内容
Pippenger 桶法	PR #486 (2017-12) 、PR #513	按标量位段分桶累加再分层合并，渐近 $O\left(\frac{256n}{\log n}\right)$ 级别的点加，大规模下远优于逐点 <code>ecmult</code>
小规模回退	PR #580、#592	scratch 空间不足时回退到无分配的 trivial 算法，保持"无运行时堆分配失败路径"的 API 语义
批量验证模块	PR #760 (早期 WIP，关闭)、PR #1107 (接口设计，关闭)、 PR #1134 (截至撰写时 open)	将上述 MSM 封装为 <code>schnorrsig</code> 批量验证 API；系数由对全部输入的哈希承诺派生，防止 rogue-key 类伪造

理论上，批量验证单个签名的边际成本随 nn 增大从一次完整 `ecmult` (约 $\sim 2^8$ 次点加量级) 降至 Pippenger 摊销下的 $O(1/\log n)$ 系数；对全节点区块链验证 (每块数千签名) 这是数量级的收益。

注：截至本报告撰写时 (2026-07)，批量验证模块在主仓库仍以 PR #1134 的形式处于评审中，其底层 `ecmult_multi` (Strauss/Pippenger + scratch 空间代价模型，近期还有 PR #1761、#1789 持续打磨) 早已是成熟基础设施。

3：数学原理与实现推导

3.1 Jacobian 投影坐标：用乘法消灭逆元

仿射坐标下点加的斜率

$$\lambda = \frac{y_2 - y_1}{x_2 - x_1}$$

需要一次域逆。即使用 `safegcd`，一次逆元的代价也约等于 50-80 次域乘，而 wNAF 主循环需要数百次点加——逆元主导一切。

Jacobian 坐标以等价类：

$$(X : Y : Z) \sim (X/Z^2, Y/Z^3)$$

表示仿射点 (x, y) ，把除法推迟到整个标量乘结束后的**唯一一次逆元** (配合 `secp256k1_ge_set_all_gej_var` 的 Montgomery 批量逆： m 个点的 Z 逆用 1 次逆元 + $3(m - 1)$ 次乘法完成)。

倍点公式推导 ($a = 0$, dbl-2009-I 族)。

设 $P = (X_1 : Y_1 : Z_1)$ ，仿射斜率

$$\lambda = \frac{3x_1^2}{2y_1}$$

代入 $x_1 = X_1/Z_1^2$, $y_1 = Y_1/Z_1^3$ 并通分：

$$\lambda = \frac{3X_1^2}{2Y_1Z_1}$$

由：

$$x_3 = \lambda^2 - 2x_1, \quad y_3 = \lambda(x_1 - x_3) - y_1$$

选取 $Z_3 = 2Y_1Z_1$ 吸收分母，得

$$X_3 = \underbrace{(3X_1^2)^2}_{\text{}} - \underbrace{8X_1Y_1^2}_{\text{}} \quad Y_3 = 3X_1^2(4X_1Y_1^2 - X_3) - 8Y_1^4 \quad Z_3 = 2Y_1Z_1$$

实现中令：

$$A = X_1^2, \quad B = Y_1^2, \quad C = B^2, \quad D = 2((X_1 + B)^2 - A - C), \quad E = 3A, \quad F = E^2$$

则：

$$X_3 = F - 2D, \quad Y_3 = E(D - X_3) - 8C$$

全程仅乘法与平方（约 $4M + 6S$ ，库内还利用 $y^2 = x^3 + 7$ 的特化进一步裁剪），零逆元。

混合加 (madd-2007-bl, $Z_2 = 1$, $7M + 4S$)

预计算表存仿射点，主循环累加器为 Jacobian。设：

$$Z_1Z_1 = Z_1^2, \quad U_2 = X_2 \cdot Z_1Z_1, \quad S_2 = Y_2 \cdot Z_1 \cdot Z_1Z_1$$

$$H = U_2 - X_1, \quad r = 2(S_2 - Y_1), \quad I = (2H)^2, \quad J = H \cdot I, \quad V = X_1 \cdot I$$

则

$$X_3 = r^2 - J - 2V, \quad Y_3 = r(V - X_3) - 2Y_1J, \quad Z_3 = (Z_1 + H)^2 - Z_1Z_1 - H^2$$

库的两个独到工程细节：

1. `secp256k1_gej` 携带 `rz` 字段追踪 Z 比率，配合 "effective affine" 技巧 (commit 4f9791a) 把 wNAF 表中点在线变换到与累加器相同的 Z ，使混合加的 U_2/S_2 重缩放被预计算摊销。
2. `gej_add_ge_var` 对 $H = 0$ 的退化分支 ($P = \pm Q$) 做了完备的代数处理而非假定不发生——这正是模块一所述统一性要求的来源。

3.2 ECDSA 与 BIP340 Schnorr：构造、线性与归约

ECDSA (d 为私钥, $P = dG$, $e = H(m)$, k 为 nonce

$$r = (kG)_x \bmod n : s = k^{-1}(e + r \cdot d) \bmod n$$

等价于：

$$k = s^{-1}e + s^{-1}r \cdot d$$

验证即检查 $(s^{-1}e)G + (s^{-1}r)P$ 的 x 坐标——这正是 `ecmult` 的原型。

注意： s 对 k 是**非线性** (含 k^{-1}) 的：两个签名的 s 无法代数合并，批量验证无解； r 依赖 kG 的 x 坐标使安全归约必须引入 "generic group + 分叉" 以外的别扭假设，且 $(r, -s)$ 可塑性迫使实现额外做低 s 归一化。

BIP340 Schnorr：\$

$$R = kG \quad (k \text{ 按偶数 } y \text{ 约定调整符号}) \quad e = H_{\text{tag}}(R_x || P_x || m) \bmod n \quad s = k + e \cdot d \bmod n$$

验证：

$$sG \stackrel{?}{=} R + eP$$

对 k 的线性带来三个结构性红利：

1. 签名方程可跨签名者/签名聚合（MuSig 模块的理论基础）；
2. 批量验证退化为一次 MSM（见 2.3 节）；
3. 存在干净的 EUF-CMA 归约。

归约梗概（ROM 下归约到离散对数）：给定 DL 挑战 P ，模拟器将 P 作为公钥。伪造者 $\mathcal{A}$ 对签名预言机的查询用**可编程随机预言机**模拟：任选 s, e ，令 $R = sG - eP$ ，设定 $H(R_x || P_x || m) = e$ ——由于 R 在编程前均匀，分布与真实不可区分。

对输出伪造 (m^*, R^*, s^*) 运行分叉引理：在 $\mathcal{A}$ 查询 $H(R^* | P | m^*)$ 处回卷并重答以 $e' \neq e$ ，以不可忽略概率获得第二伪造 s' 。两个伪造满足

$$s = k + ed, \quad s' = k + e'd \implies d = \frac{s - s'}{e - e'} \bmod n$$

直接解出离散对数。归约代价为分叉引理固有的平方损失 q_H/ϵ ，这是“紧归约 vs 具体安全”权衡的标准注脚。

ECDSA 不存在如此简洁的归约（已知结果需要更强假设或 generic group model），这是 BIP340 在理论侧的核心理由之一。

3.3 Tagged Hash 的数学动机

BIP340 定义：

$$\text{TaggedHash}(\text{tag}, m) = \text{SHA256}(\text{SHA256}(\text{tag}) || \text{SHA256}(\text{tag}) || m)$$

三层动机：

1. **域分离 (domain separation)**：若同一哈希函数 H 在挑战 e 、nonce 派生、辅助随机数等多个上下文中复用，跨协议的“意外共谋”（某协议的消息恰好是另一协议的合法哈希输入）会破坏模拟器中随机预言机的可编程性——归约要求每个上下文的 H 是**独立**随机预言机。以 $\text{SHA256}(\text{tag})$ 作为有效 IV，等价于为每个 tag 实例化一条独立的 Merkle-Damgård 链，把单一哈希函数分裂为一族前缀无关（prefix-free）的函数族。前缀自由性也一并消除了长度扩展类混淆：不同 tag 的输出不可能互为 MD 链的延续状态。
2. **抗长度扩展的直接性**：朴素做法 $H(\text{tag} || m)$ 中 tag 长度可变时存在 $(\text{tag}', m') \neq (\text{tag}, m)$ 的拼接歧义；定长 64 字节的双前缀块使编码无歧义。
3. **Mid-state 性能**：由于前缀 $\text{SHA256}(\text{tag}) || \text{SHA256}(\text{tag})$ 恰为一个完整 64 字节压缩块，其实现可缓存压缩函数在该块后的中间状态（mid-state），之后每次 tagged hash 只对 m 做一次（或少数次）压缩。`schnorrsig` 模块中挑战哈希对同一 tag 被调用无数次，此优化省掉每次调用约一半的压缩轮——这在 `src/modules/schnorrsig/main_impl.h` 中以预初始化 sha256 上下文的形式实现（近期 PR #1825 还在精简这条路径上的冗余初始化）。

4. 关键 Commit / PR 索引

编号	描述	影响域
commit 399c03f	"Make endomorphism optimization optional" (2014)	GLV 默认关闭
PR #20	引入 GLV endomorphism 支持	验证性能

编号	描述	影响域
PR #57 / #190	引入标量盲化与投影盲化	签名安全 (DPA 防御)
commit 5de4c5d	修复 $P+(-\lambda)PP+(-\lambda)P$ 退化情形统一性	常数时间群运算
PR #486 / #513	Pippenger 桶法 MSM	批量验证性能
PR #558	引入 BIP-340 Schnorr 签名模块	功能扩展
PR #830	删除非 endomorphism 代码路径	验证端统一使用 GLV
PR #831	safegcd 模逆替代 GMP, 移除 libgmp 依赖	安全 + 性能
PR #844	schnorrsig API 大改	接口稳定性
PR #906	divsteps 常数时间修正	常数时间模逆
PR #1058	signed-digit multi-comb 重写 <code>ecmult_gen</code>	签名性能
PR #1134	Schnorr 批量验证 API (open)	批量验证
PR #1446	删除 x86_64 汇编 (field_5x52_asm)	可审计性
PR #1564	默认 comb 表扩大至 86 KiB	签名性能
PR #1761 / #1789	scratch 空间代价模型打磨	<code>ecmult_multi</code> 鲁棒性
PR #1825	tagged hash mid-state 初始化精简	哈希性能

结语

secp256k1 的十年演进呈现一条一以贯之的主线：**按秘密性分层准入优化**。

- **验证路径**吸收 GLV、wNAF、Pippenger 等一切代数红利 (PR #20、#830、#486) ；
- **签名路径**则以 comb 固定访问、cmov 掩码、标量/投影盲化和 safegcd 常数时间逆元构筑纵深防御 (PR #57、#190、#831、#1058) ；
- 并以穷举测试与 Valgrind 秘密标记把"常数时间"从承诺变为 CI 中可证伪的断言。

汇编的退场 (PR #1446) 与批量验证的漫长评审 (PR #1134) 则提醒我们：**在这个仓库里，可审计性与正确性证据永远排在微基准数字之前**。